

UITax — Unemployment Insurance Tax Microservices System

Project Documentation — Prepared for Vinay Manavarthi

July 2026

Contents

1	1. Introduction — What Is This Project	4
1.1	1.1 In one sentence	4
1.2	1.2 Why this project exists	4
1.3	1.3 Who built this and why	4
2	2. A Little Bit About the Domain	5
2.1	2.1 What is Unemployment Insurance Tax?	5
2.2	2.2 How this maps to the project, in plain terms	5
3	3. Who This Document Is For / Prerequisites	6
3.1	3.1 Recommended background knowledge	6
4	4. Glossary of Terms	7
5	5. High-Level Design (HLD)	10
5.1	5.1 Architecture Diagram	10
5.2	5.2 The Five Services, In Plain English	10
5.3	5.3 The Overall Data Flow, Narrated	11
6	6. Low-Level Design (LLD)	12
6.1	6.1 tax-filing-service	12
6.2	6.2 payment-service	12
6.3	6.3 notification-service	13
6.4	6.4 api-gateway	13
6.5	6.5 eureka-server	13
7	7. Technologies and Tools Used	14
7.1	7.1 Core Language & Framework	14
7.2	7.2 Data & Messaging	14
7.3	7.3 Resilience & Cross-Cutting Concerns	14
7.4	7.4 Infrastructure & Tooling	15
8	8. Environment Setup	16
8.1	8.1 Prerequisites to install	16
8.2	8.2 Enabling WSL2 (required for Docker Desktop on Windows)	16
8.3	8.3 Bringing up the local infrastructure	16
8.4	8.4 A note on file extensions (a real trap hit repeatedly in this project)	17
8.5	8.5 Running each Spring Boot service	17
9	9. Database Schema	18
9.1	9.1 Schema Diagram	18
9.2	9.2 filingdb (PostgreSQL, port 5433)	18
9.3	9.3 paymentdb (PostgreSQL, port 5434)	19
9.4	9.4 Why two completely separate databases?	20
10	10. Core Concepts Explained In Detail	21
10.1	10.1 Microservices Architecture	21

10.2	10.2 Service Discovery (Eureka)	21
10.3	10.3 API Gateway	21
10.4	10.4 The Dual-Write Problem and the Transactional Outbox Pattern	22
10.5	10.5 Apache Kafka Basics	23
10.6	10.6 Idempotency	25
10.7	10.7 Circuit Breaker (Resilience4j)	26
10.8	10.8 Dead Letter Topic (DLT)	26
10.9	10.9 JPA Auditing	27
10.10	10.10 Bean Validation	28
11	11. Important Code Snippets	30
11.1	11.1 The TaxFiling Entity	30
11.2	11.2 The OutboxEvent Entity	31
11.3	11.3 The Circuit Breaker Self-Invocation Fix	32
11.4	11.4 Custom Circuit Breaker State-Change Logging	32
11.5	11.5 application.yml Reference — Key Sections Per Service	33
12	12. Issues Encountered During Development, and How Each Was Resolved	35
12.1	Issue 1 — Docker Desktop failed to start: WSL not installed	35
12.2	Issue 2 — docker-compose.yml silently saved as docker-compose.yml.txt	35
12.3	Issue 3 — application.yml never actually existed; services silently used defaults	35
12.4	Issue 4 — Eclipse showed pom.xml errors that were not real errors	35
12.5	Issue 5 — Stale processes held onto ports after a service was “stopped”	36
12.6	Issue 6 — TaxFiling entity was missing its constructor and getters	36
12.7	Issue 7 — jakarta.validation imports unresolved	36
12.8	Issue 8 — YAML indentation mistake silently broke a new config section	36
12.9	Issue 9 — Spring Cloud version incompatible with Spring Boot version	37
12.10	Issue 10 — PowerShell silently stripped Spring placeholders from a config value	37
12.11	Issue 11 — Eureka registered services under an illegal hostname	37
12.12	Issue 12 — notification-service failed to register with Eureka	38
12.13	Issue 13 — The circuit breaker was configured correctly but never actually engaged	38
12.14	Issue 14 — A two-part ObjectMapper / date deserialization bug	38
12.15	Issue 15 — Permanently-failing messages were silently discarded	39
13	13. Testing Strategy — How Each Feature Was Verified	40
13.1	13.1 Basic end-to-end flow	40
13.2	13.2 Outbox pattern	40
13.3	13.3 Multiple independent consumers	40
13.4	13.4 Idempotency	40
13.5	13.5 Circuit breaker	40
13.6	13.6 Dead Letter Topic	41
13.7	13.7 Service discovery and gateway routing	41
14	14. Project Status and Future Improvements	42
14.1	14.1 What is fully working and verified	42
14.2	14.2 Known limitations and suggested future work	42
15	15. How to Run This Project Locally — Quick Reference Runbook	43

16	16. Appendix	44
16.1	16.1 Full docker-compose.yml (Shared Infrastructure)	44
16.2	16.2 Full application.yml — tax-filing-service	45
16.3	16.3 Full application.yml — payment-service	46
16.4	16.4 Full application.yml — notification-service	47
16.5	16.5 Full application.yml — api-gateway	48
16.6	16.6 Full application.yml — eureka-server	48
17	Closing Note	50

1 1. Introduction — What Is This Project

UITax is a hands-on learning project that simulates a small part of a real government system: **filing un-employment insurance tax and paying it**. It is built the way modern backend systems at real companies (banks, payment processors, government IT departments) are built today — as a set of small, independent programs called **microservices**, instead of one giant application.

1.1 1.1 In one sentence

An employer submits a tax filing through a web request; the system safely records it, tells other parts of the system that it happened, processes a payment for it, and logs a notification — all without any single piece of the system ever needing to trust that another piece is always available.

1.2 1.2 Why this project exists

The purpose of this project is **not** to be a production tax system for any real government agency. It is a **personal learning project** built to gain real, hands-on, interview-ready experience with technologies and patterns that are hard to learn from reading alone:

- Building more than one Spring Boot service and making them talk to each other
- Apache Kafka (event-driven communication between services)
- Service discovery (so services can find each other without hardcoded addresses)
- An API Gateway (a single front door for all requests)
- The **Transactional Outbox** pattern (a reliability pattern used at real fintech companies)
- **Idempotency** (never processing the same event twice)
- **Circuit breakers** (protecting a service from a failing dependency)
- **Dead Letter Topics** (never silently losing a message that cannot be processed)

1.3 1.3 Who built this and why

This project was built by a Java Full Stack Developer with several years of Spring Boot experience, specifically to close a gap: strong single-application (monolith) skills, but limited hands-on experience with the microservices tooling that many mid-to-senior Java job postings now expect (Kafka, Eureka, API Gateway, resilience patterns). Every part of the system was built, broken, debugged, and fixed by hand — the “Issues Encountered” section later in this document is a faithful log of the real problems hit along the way, because those problems (and how they were diagnosed) are often the most valuable part of the learning.

2 2. A Little Bit About the Domain

Before diving into the technology, it helps to understand **what the system is actually pretending to do** in the real world.

2.1 2.1 What is Unemployment Insurance Tax?

In the United States, most employers are required to pay a tax that funds **unemployment insurance** — the program that pays temporary income to people who lose their jobs through no fault of their own. This is separate from the tax an employee sees taken out of their own paycheck; it is a tax the **employer** pays, based on the wages they paid their employees during a period (usually a calendar quarter, e.g. “2026-Q2”).

In a real state Department of Labor, the general real-world flow looks like this:

1. An employer reports the total wages they paid their employees for a quarter (a **filing**).
2. Based on those wages and the employer’s tax rate, a **tax amount due** is calculated.
3. The employer pays that amount.
4. The state confirms the payment and updates its records.
5. Various downstream systems (accounting, notifications, compliance reporting) need to know a filing happened.

2.2 2.2 How this maps to the project, in plain terms

Real-world concept	This project’s equivalent
Employer reports quarterly wages	POST /api/filings request to tax-filing-service
State records the filing	A row saved in the tax_filing table
State needs to tell other departments “a filing happened”	An event published to Kafka
Payment processing department charges/settles tax due	payment-service consuming that event
Notifications team alerts relevant people	notification-service consuming that same event

The project intentionally keeps the *business* logic very simple (there is no real payment gateway, no real tax rate calculation, no authentication) so that all of the *engineering effort* can go into the microservices patterns themselves, which is the actual point of the exercise.

3 3. Who This Document Is For / Prerequisites

This document is written so that **two different kinds of readers** can get value from it:

- **A non-technical reader** (e.g. a recruiter, a hiring manager skimming a portfolio, a curious friend) should be able to read the plain-English explanations in each section and understand *what* the system does and *why* each piece exists, without needing to read any code.
- **A technical reader** (a fellow developer, an interviewer, someone trying to learn microservices themselves) gets the full detail underneath: real configuration, real code, real error messages, and real fixes.

3.1 3.1 Recommended background knowledge

To get the most out of the technical sections, it helps (but is not strictly required) to already know:

- Basic Java syntax and object-oriented programming
- The basics of Spring Boot (what a `@RestController`, `@Service`, and `@Entity` are)
- Basic SQL and what a relational database is
- What a REST API is (GET, POST, JSON)
- Comfort using a command line / terminal (PowerShell, in this project's case, since development was done on Windows)

No prior knowledge of Kafka, microservices, Docker, or any of the resilience patterns is assumed. Every one of those concepts is explained from first principles in Section 10.

4 4. Glossary of Terms

This glossary is written for **quick lookup** — if a term in a later section is unfamiliar, it is defined here in plain English first, with the more technical detail second.

Microservice — A small, independently-runable program that does one job well (e.g., “handle tax filings”) and talks to other microservices over the network, instead of one large program doing everything.

Monolith — The opposite of a microservice architecture: one single application containing all the functionality. Most developers start their careers working on monoliths, which is why microservices patterns can initially feel unfamiliar.

REST API — A common style of building web services where a client sends HTTP requests (GET, POST, PUT, DELETE) to specific URLs to read or change data, and the server responds with data, usually formatted as JSON.

JSON — A simple, human-readable text format for representing structured data, e.g. `{"employerId": "abc-123", "taxDue": 4500.00}`. It is the standard format used for data sent between services in this project.

Database (relational) — A structured place to permanently store data in tables made of rows and columns. This project uses **PostgreSQL**.

Database-per-service — A microservices principle where each service owns and exclusively accesses its own database; no two services are ever allowed to directly query each other’s database tables.

JPA / Hibernate — JPA (Jakarta Persistence API) is a Java standard for mapping Java objects to database tables. Hibernate is the specific library (implementation of that standard) Spring Boot uses to actually talk to the database on your behalf, so you rarely have to write raw SQL.

Entity — A Java class annotated `@Entity` that represents one row of one database table.

DTO (Data Transfer Object) — A small, simple Java class or record used only to define the exact shape of data going into or out of an API, kept deliberately separate from the database `@Entity` classes.

UUID (Universally Unique Identifier) — A randomly generated 128-bit identifier (e.g. `f47ac10b-58cc-4372-a567-0e02b2c3d479`) used as a primary key in this project instead of an auto-incrementing number, because it can be generated independently by any service, in memory, with no risk of collision.

Transaction (ACID) — A group of database operations that are guaranteed to either **all** succeed together or **all** be undone together, with nothing left half-done.

Docker / Container — Docker lets you package and run a piece of software (like a database or a message broker) in an isolated, portable “container” without installing it directly on your computer. This project uses Docker to run PostgreSQL, Kafka, and related tools.

Docker Compose — A tool for describing and starting *multiple* Docker containers together, defined in one `docker-compose.yml` file.

Apache Kafka — A high-throughput messaging system that lets one program publish messages to a named **topic**, and lets any number of other programs independently read those messages, at their own pace, without the messages being deleted after one program reads them.

Topic — A named category/log inside Kafka that messages are published to and read from (e.g. `tax-filing-events`).

Partition — Kafka splits a topic into one or more partitions for scalability. Messages within one partition are strictly ordered.

Producer — Any piece of code that sends (publishes) messages to a Kafka topic.

Consumer — Any piece of code that reads (subscribes to) messages from a Kafka topic.

Consumer Group — A named identity a consumer registers under. Kafka tracks a separate reading position (offset) *per consumer group*, meaning two different consumer groups reading the same topic never interfere with each other — each gets its own full copy of every message.

Offset — A number marking a consumer group’s current position (“bookmark”) within a partition’s message log.

Idempotency — Designing a system so that processing the same message or request more than once has no additional effect beyond processing it once (e.g., never charging someone twice for one event).

Service Discovery — A mechanism that lets services find each other’s network addresses dynamically at runtime by asking a shared directory, instead of hardcoding IP addresses and ports.

Eureka — The specific service discovery tool (originally built by Netflix, provided via Spring Cloud) used in this project.

API Gateway — A single entry point that receives all incoming client requests and forwards (“routes”) each one to the correct backend service.

Load Balancing — Distributing incoming requests across multiple available instances of a service, so no single instance is overwhelmed. The `lb://` prefix used in this project’s gateway configuration signals “look this service name up via service discovery and load-balance across whatever instances exist.”

Circuit Breaker — A safety mechanism that stops a service from repeatedly calling a dependency that has been failing, giving that dependency time to recover instead of being hammered with requests it cannot handle.

Resilience4j — The Java library used in this project to implement the circuit breaker pattern.

Dual-Write Problem — The reliability problem that occurs when a piece of code needs to update a database *and* separately notify another system (like Kafka) about the same event — because these are two independent operations, one can succeed while the other silently fails.

Transactional Outbox Pattern — The reliability pattern used in this project to solve the dual-write problem: instead of calling Kafka directly, an event is written as a database row in the *same* transaction as the actual business data; a separate background process later reads and publishes those rows to Kafka.

Dead Letter Topic (DLT) — A separate Kafka topic used to hold messages that a consumer tried and permanently failed to process, so that they are preserved for investigation instead of being silently discarded.

AOP (Aspect-Oriented Programming) / Proxy — A technique Spring uses to add extra behavior (like starting a transaction, or applying a circuit breaker) around a method, by wrapping the real object in an invisible “proxy” object. This only works when the method is called *from a different class* — calling it on *this*, from within the same class, bypasses the proxy entirely. This exact issue was hit and fixed in this project (see Issue #13).

YAML — A human-readable text format (`application.yml`) used to configure Spring Boot applications. It represents nested configuration using indentation instead of repeated dotted prefixes.

Bean (Spring Bean) — Any object that the Spring Framework creates and manages on your behalf, so you can have it automatically injected into other classes that need it, rather than constructing it yourself.

Actuator — A Spring Boot module that exposes built-in HTTP endpoints (e.g. `/actuator/health`, `/actuator/circuitbreakers`) for monitoring and inspecting a running application.

5. High-Level Design (HLD)

5.1 Architecture Diagram

Figure 1. High-Level System Architecture

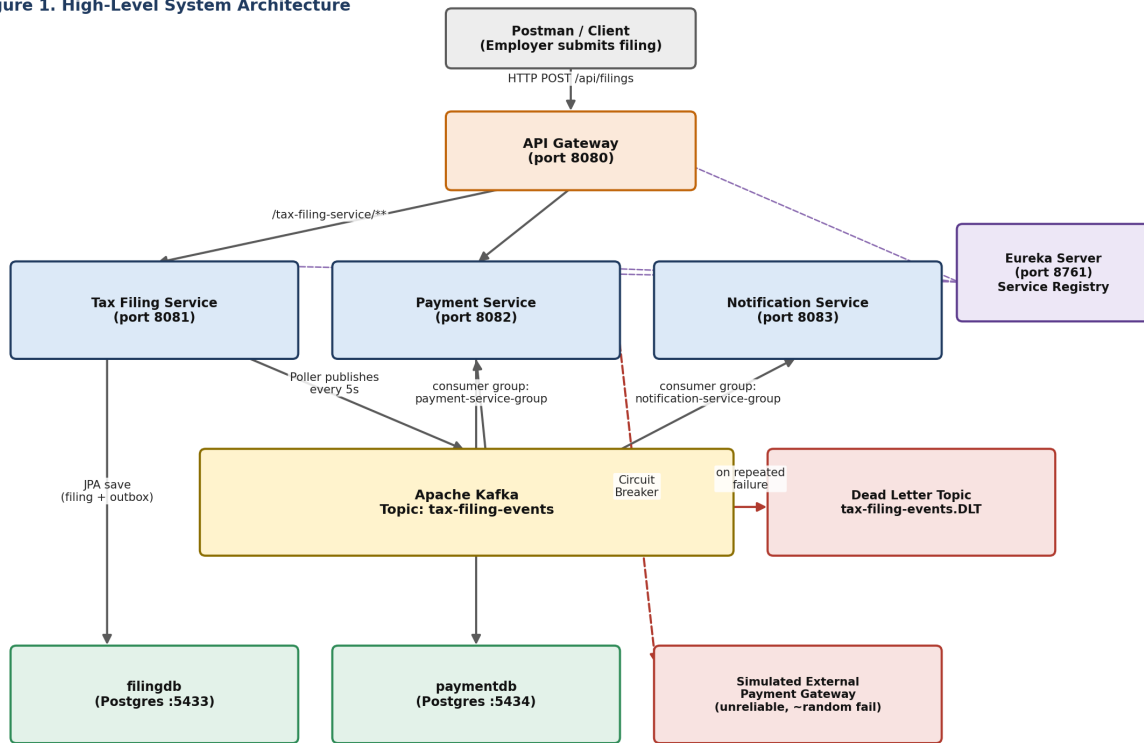


Figure 1: Figure 1. High-Level System Architecture

5.2 The Five Services, In Plain English

Service	Port	What it actually does
eureka-server	8761	A phone book. Every other service checks in here on startup and says “I’m alive, here’s my address.” Other services ask it “where is payment-service right now?” instead of hardcoding an address.
api-gateway	8080	The single front door. A client calls this one address; the gateway figures out (via Eureka) which real service should handle the request, and forwards it there.

Service	Port	What it actually does
tax-filing-service	8081	Where an employer's filing is submitted. Saves the filing to its own database and safely announces "a filing happened" so other services can react.
payment-service	8082	Listens for "a filing happened" announcements and processes a (simulated) payment for it. Makes sure it never processes the same filing twice, and protects itself from a flaky external payment gateway.
notification-service	8083	Also listens for "a filing happened" announcements, and simply logs it — the simplest service in the system, built to demonstrate that multiple independent services can react to the same event.

5.3 5.3 The Overall Data Flow, Narrated

1. An employer (via Postman, or in a real system, a web form) sends a **POST** request containing filing details to the **API Gateway**.
2. The gateway asks **Eureka**: "who is tax-filing-service right now?" and forwards the request there.
3. **tax-filing-service** validates the request, then — in a single database transaction — saves the filing **and** a note-to-self saying "an event needs to be published," to its own database (**filingdb**).
4. A background job inside **tax-filing-service**, running every 5 seconds, finds that note-to-self and publishes an event to a Kafka topic called **tax-filing-events**. It only marks the note as "done" once Kafka has confirmed the message was received.
5. **payment-service** and **notification-service** are both independently subscribed to **tax-filing-events**, under two different consumer group names. Kafka delivers a full, independent copy of the event to each of them.
6. **payment-service** checks whether it has already processed this exact event (idempotency check), then attempts to call a (simulated, unreliable) external payment gateway — protected by a circuit breaker — to process the payment, and saves the result to its own database (**paymentdb**).
7. **notification-service** simply logs that the event was received.
8. If **payment-service** cannot successfully process an event after several retries (e.g., the simulated gateway is down and the circuit breaker is open), the event is republished to a separate **Dead Letter Topic** instead of being silently lost.

No two services ever share a database. No service ever hardcodes another service's network address. Every step above is described and demonstrated in detail later in this document.

6 6. Low-Level Design (LLD)

This section goes one level deeper than the HLD: the actual package structure, entity design, and key classes inside each service.

6.1 6.1 tax-filing-service

Package root: `com.uitax.taxfiling`

```
com.uitax.taxfiling
├─ controller      (TaxFilingController)
├─ service         (TaxFilingService interface)
├─ service.impl    (TaxFilingServiceImpl, OutboxEventPublisher)
├─ repository      (TaxFilingRepository, OutboxEventRepository)
├─ entity          (TaxFiling, FilingStatus, OutboxEvent, Auditable)
├─ dto             (TaxFilingRequest, TaxFilingResponse)
├─ event           (TaxFilingSubmittedEvent)
├─ config          (JpaAuditingConfig, AuditorAwareImpl)
└─ exception       (reserved for centralized error handling – see Section 14)
```

Design decisions worth noting:

- `TaxFiling` never exposes a `setStatus()` method. Instead, it exposes intention-revealing methods, `markProcessed()` and `markFailed()`, so that status can only change through defined, valid transitions — not through arbitrary external mutation.
- The REST layer (`TaxFilingController`) never accepts or returns the `TaxFiling` entity directly. It only ever deals in `TaxFilingRequest` / `TaxFilingResponse`, so the public API contract is completely decoupled from the internal database structure.
- `TaxFilingServiceImpl.submitFiling()` is the single most important method in the service — it is where the transactional outbox write happens (see Section 10.4 for the full explanation).
- `OutboxEventPublisher` is the poller responsible for actually delivering outbox rows to Kafka (see Section 10.4).

6.2 6.2 payment-service

Package root: `com.uitax.payment`

```
com.uitax.payment
├─ controller
├─ service
├─ service.impl    (PaymentEventListener, PaymentGatewayClient, PaymentGatewayInvoker)
├─ repository      (PaymentRepository, ProcessedEventRepository)
├─ entity          (Payment, PaymentStatus, ProcessedEvent)
├─ dto
├─ event           (TaxFilingSubmittedEvent – its own copy, see Section 10.5)
├─ config          (JacksonConfig, CircuitBreakerConfig, KafkaErrorHandlingConfig)
└─ exception
```

Design decisions worth noting:

- `ProcessedEvent` is intentionally a minimal entity with no business fields at all — its only job is answering “have I seen this event before?” This is what makes idempotency possible (Section 10.6).

- `PaymentGatewayClient` and `PaymentGatewayInvoker` are **deliberately two separate classes**, not one. This split exists specifically so that Resilience4j's `@CircuitBreaker` annotation actually works — see Issue #13 in Section 12 for the full story of why this split was necessary.
- `Payment.filingId` has a **unique** database constraint, providing a second, independent layer of duplicate protection beyond the `ProcessedEvent` table.

6.3 6.3 notification-service

Package root: `com.uitax.notification`

```
com.uitax.notification
└─ service                (NotificationEventListener)
```

This is deliberately the thinnest service in the project — a single `@KafkaListener` method that logs the event it receives. It exists to prove that Kafka correctly delivers the same event independently to multiple, unrelated consumers (see Section 10.5).

6.4 6.4 api-gateway

Package root: `com.uitax.gateway`

This service contains almost no custom Java code — its behavior is defined almost entirely through `application.yml` route configuration (Section 11 shows the full file). It uses the **Spring Cloud Gateway MVC** variant (`spring-cloud-starter-gateway-server-webmvc`) rather than the traditional reactive Gateway, which keeps it consistent with the rest of the project's Servlet/Tomcat-based stack rather than introducing a reactive (WebFlux/Netty) runtime just for this one service.

6.5 6.5 eureka-server

Package root: `com.uitax.eureka`

A single `@SpringBootApplication` class annotated `@EnableEurekaServer`. All of its actual behavior comes from `application.yml`.

7 7. Technologies and Tools Used

7.1 7.1 Core Language & Framework

Technology	Version	Purpose
Java	25	Programming language
Spring Boot	3.5.16 (eureka-server, tax-filing-service, api-gateway, notification-service)	Application framework
Spring Boot	4.1.0 (payment-service)	Application framework
Spring Cloud	2025.0.0 (paired with Boot 3.5.x services)	Netflix Eureka client/server, Gateway
Spring Cloud	2025.1.2 (paired with Boot 4.x, payment-service)	Netflix Eureka client, Resilience4j integration
Maven	(bundled with Spring Boot parent POM)	Build tool and dependency management

A deliberate note on the mixed Spring Boot versions: payment-service ended up on a newer Spring Boot line (4.1.0) than the other four services (3.5.16), simply because of when each project was generated via Spring Initializr. Rather than “fixing” this retroactively, it was left as-is, because it accurately reflects a very real situation in actual microservices organizations: different services evolve and upgrade independently, on their own schedules, and are not always on identical framework versions. Learning to pair the *correct* Spring Cloud release train with *each* service’s own Spring Boot version (see Issue #9) is itself one of the valuable lessons from this project.

7.2 7.2 Data & Messaging

Technology	Version	Purpose
PostgreSQL	16	Relational database (two independent instances)
Apache Kafka	(Confluent Platform 7.6.1 images)	Event streaming / messaging
Zookeeper	(Confluent Platform 7.6.1)	Kafka’s coordination service (this project uses the traditional Zookeeper-based Kafka setup rather than the newer KRaft mode)
Hibernate / JPA	(via spring-boot-starter-data-jpa)	Object-relational mapping

7.3 7.3 Resilience & Cross-Cutting Concerns

Technology	Purpose
Resilience4j	Circuit breaker implementation

Technology	Purpose
Spring Kafka's <code>DefaultExceptionHandler</code> + <code>DeadLetterPublishingRecoverer</code>	Dead Letter Topic support
Jakarta Bean Validation (spring-boot-starter-validation)	Request payload validation (@NotNull, @NotBlank, @PositiveOrZero)
Spring Boot Actuator	Health checks and circuit breaker inspection endpoints
SLF4J / Logback	Application logging

7.4 7.4 Infrastructure & Tooling

Tool	Purpose
Docker Desktop (with WSL2 backend)	Running PostgreSQL, Kafka, Zookeeper, and Kafka UI locally on Windows
Docker Compose	Declaring and starting the entire local infrastructure stack with one command
Kafka UI (provectuslabs/kafka-ui)	A web dashboard for inspecting Kafka topics, messages, and consumer groups
Eclipse IDE (with Spring Tools)	Primary development environment
Postman	Manual REST API testing
DBeaver	Visual inspection of both PostgreSQL databases
PowerShell	Command-line environment on Windows, used for all file/process management shown in this document

8. Environment Setup

This section is a step-by-step runbook for setting up this project's environment from a fresh Windows machine, including the real obstacles that were hit.

8.1 Prerequisites to install

1. **Java Development Kit (JDK) 25** — required to compile and run all five services.
2. **Eclipse IDE** with the Spring Tools plugin — used for all development in this project.
3. **Docker Desktop for Windows** — used to run PostgreSQL, Kafka, Zookeeper, and Kafka UI without installing them directly on Windows.
4. **Postman** — for manually testing REST endpoints.
5. **DBeaver** — for visually inspecting the two PostgreSQL databases.

8.2 Enabling WSL2 (required for Docker Desktop on Windows)

Docker Desktop on Windows runs Linux containers through the **Windows Subsystem for Linux (WSL2)**. On a machine where this had never been set up, Docker Desktop failed on first launch with:

```
Docker Desktop - WSL not installed
Windows Subsystem for Linux (WSL) is not installed.
```

Fix, run from an **administrator** PowerShell window:

```
wsl --install
```

This downloads and installs WSL2 plus a default Ubuntu Linux distribution. After it finishes, **a full machine restart is required** before Docker Desktop will recognize WSL2 as available. On first boot after installing, Ubuntu will prompt for a new Linux username and password — this is only used internally by WSL and is never touched directly for this project.

8.3 Bringing up the local infrastructure

All shared infrastructure (databases, Kafka) is defined in one `docker-compose.yml`, kept in a folder separate from any single service's source code (e.g. `C:\uitax\infra`), since it is shared across the whole project, not owned by one service.

The full file is provided in the Appendix (Section 16.1). To start everything:

```
cd C:\uitax\infra
docker compose up -d
docker compose ps
```

`docker compose ps` should show five running containers: `zookeeper`, `kafka`, `kafka-ui`, `postgres-filing`, `postgres-payment`.

Verify Kafka UI is reachable: open `http://localhost:8090` in a browser — it should show a cluster named `local` as online.

Verify each Postgres instance is reachable:

```
docker exec -it postgres-filing psql -U filinguser -d filingdb -c "\dt"
docker exec -it postgres-payment psql -U paymentuser -d paymentdb -c "\dt"
```

8.4 8.4 A note on file extensions (a real trap hit repeatedly in this project)

Windows Explorer hides known file extensions by default. Several times during this project, a file that was intended to be `docker-compose.yml` or `application.yml` was actually saved as `docker-compose.yml.txt` or `application.yml.txt` without it being obvious from Explorer's file listing. This caused Docker and Spring Boot to silently fail to find the expected configuration.

The reliable way to check a filename on Windows is PowerShell's `dir` command, which always shows the true filename:

```
dir
```

If a hidden `.txt` extension is found, it can be fixed directly:

```
Rename-Item "application.yml.txt" "application.yml"
```

8.5 8.5 Running each Spring Boot service

Each of the five services is a separate Eclipse project. They must be started in a specific order, because each one (except `eureka-server` itself) needs to successfully register with Eureka, and `api-gateway` specifically needs the other services to already be registered so its routing table has somewhere to point to:

1. `eureka-server` — wait for `Started EurekaServerApplication...` and confirm `http://localhost:8761` loads.
2. `tax-filing-service`
3. `payment-service`
4. `notification-service`
5. `api-gateway` — started last, since it depends on the others already being discoverable.

In Eclipse, each is started via **right-click the project** → **Run As** → **Spring Boot App**.

9 Database Schema

9.1 Schema Diagram

Figure 5. Database Schema — Two Independent Databases (Database-per-Service)

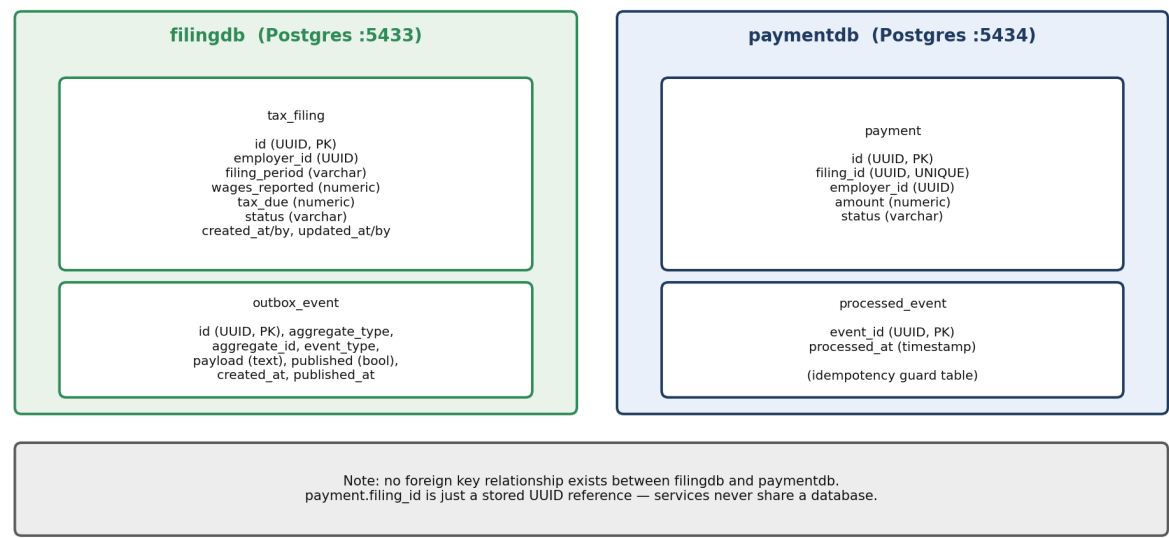


Figure 2: Figure 5. Database Schema

9.2 9.2 filingdb (PostgreSQL, port 5433)

Owned exclusively by tax-filing-service.

Table: tax_filing

Column	Type	Notes
id	uuid	Primary key, generated in application memory (GenerationType.UUID)
employer_id	uuid	References an employer conceptually — no foreign key, since employer data belongs to a different bounded context

Column	Type	Notes
filing_period	varchar	e.g. "2026-Q2"
wages_reported	numeric(38,2)	Stored as BigDecimal in Java — never double, to avoid floating-point rounding errors with money
tax_due	numeric(38,2)	Same reasoning as above
status	varchar, with a database CHECK constraint	Restricted to SUBMITTED, PROCESSED, FAILED — enforced both in the Java enum and, automatically, at the database level
created_at, created_by, updated_at, updated_by	timestamp / varchar	Populated automatically by Spring Data JPA Auditing (Section 10.9)

Table: outbox_event

Column	Type	Notes
id	uuid	Primary key
aggregate_type	varchar	e.g. "TaxFiling" — which kind of business object this event describes
aggregate_id	uuid	Which specific record (matches tax_filing.id)
event_type	varchar	e.g. "TaxFilingSubmitted"
payload	text	The full JSON event body, exactly as it will be sent to Kafka
published	boolean	false until the background poller successfully sends it to Kafka
created_at	timestamp	When the row was written
published_at	timestamp, nullable	When it was actually delivered to Kafka; null until then

9.3 9.3 paymentdb (PostgreSQL, port 5434)

Owned exclusively by payment-service.

Table: payment

Column	Type	Notes
id	uuid	Primary key
filing_id	uuid, UNIQUE	The uniqueness constraint is a deliberate second layer of duplicate-payment protection

Column	Type	Notes
employer_id	uuid	Copied from the incoming event
amount	numeric(38,2)	The tax amount charged
status	varchar	PROCESSED or FAILED

Table: processed_event

Column	Type	Notes
event_id	uuid	Primary key — not auto-generated ; it is the incoming event's own identifier, used purely to answer “have I seen this before?”
processed_at	timestamp	When it was recorded as processed

This table has no other columns and no business meaning of its own — its only job is idempotency (Section 10.6).

9.4 9.4 Why two completely separate databases?

This is a deliberate microservices principle called **database-per-service**. `filingdb` and `paymentdb` share no tables, no foreign keys, and no direct queries between them. `payment.filing_id` is *just a stored value* — a plain UUID copied from an event — never a real, enforced foreign key relationship, because `payment-service` is never allowed to reach directly into `tax-filing-service`'s database. If two services could freely query each other's tables, they would no longer be independently deployable or independently scalable, which defeats the purpose of splitting them into separate services in the first place.

10 10. Core Concepts Explained In Detail

Each concept below is explained twice: first in **plain English**, then with the **technical detail** of exactly how it was implemented in this project.

10.1 10.1 Microservices Architecture

In plain English: Instead of building one large program that does everything, the system is split into several small programs, each responsible for one job, that talk to each other over the network. This is similar to a company having separate departments (filing, payments, notifications) instead of one person doing every job — each department can be staffed, changed, or scaled independently, as long as they agree on how to communicate with each other.

Technically: Each of the five services in this project is its own Spring Boot application, with its own `pom.xml`, its own port, its own (where applicable) database, and its own deployable JAR. They communicate through two mechanisms only: synchronous HTTP calls (routed through the API Gateway) and asynchronous Kafka events. No service ever imports another service's Java code or directly queries another service's database.

10.2 10.2 Service Discovery (Eureka)

In plain English: Imagine every employee at a company had to memorize every other employee's phone number by heart, and update it manually whenever someone changed desks. That would break constantly. Instead, most companies just have a directory — you look up a name, and you get the current number. Eureka is that directory for services.

Technically: `eureka-server` runs as its own Spring Boot application (`@EnableEurekaServer`) on port 8761. Every other service includes `spring-cloud-starter-netflix-eureka-client` and, on startup, automatically registers itself — announcing its application name and network address. `api-gateway` uses this registry to resolve `lb://tax-filing-service` and `lb://payment-service` into real, currently-available addresses at request time, rather than having them hardcoded in configuration.

A real complication was hit here: on Windows, a machine's hostname can contain characters (like an underscore, as in `Vinay_Laptop`) that are technically illegal in a URI. This caused Eureka to register services under an address the Java URI class refused to parse. The fix — forcing `eureka.instance.hostname` to `localhost` and an explicit `instance-id` — is documented in full as Issue #11 in Section 12.

10.3 10.3 API Gateway

In plain English: Instead of a client needing to know the separate addresses of five different services, it just talks to one address — the gateway — and the gateway quietly figures out where to actually send the request.

Technically: This project uses **Spring Cloud Gateway's MVC (Servlet-based) variant** (`spring-cloud-starter-gateway-server-webmvc`), configured entirely through YAML routes. Each route defines a Path predicate (which incoming URLs it matches), a `uri` (using the `lb://` scheme to route via Eureka-based load balancing), and a `StripPrefix=1` filter (which removes the leading path segment, e.g. `/tax-filing-service`, before forwarding the request, since the real service's controller has no such prefix in its own mapping).

10.4 10.4 The Dual-Write Problem and the Transactional Outbox Pattern

In plain English: Imagine finishing writing a letter and personally running it to the mailbox before doing anything else. If the mailbox happens to be jammed that day, you're stuck. Now imagine instead you just drop the letter in an outgoing tray on your desk, and a mail carrier reliably swings by every few minutes to collect and mail anything waiting there. If the carrier is running late one round, your letter simply waits safely — it is never lost, and you were never blocked waiting for the mailbox.

Technically: If tax-filing-service saved a filing to its database and then *immediately* called Kafka to publish an event, those would be two independent operations against two independent systems. If the database save succeeds but the Kafka call fails (network blip, broker restart, anything), the filing would exist with no record anyone was ever told about it — a silent, permanent gap. This is called the **dual-write problem**.

The fix implemented here is the **Transactional Outbox pattern**:

```
@Transactional
public TaxFilingResponse submitFiling(TaxFilingRequest request) {

    TaxFiling filing = new TaxFiling(
        request.employerId(), request.filingPeriod(),
        request.wagesReported(), request.taxDue());
    taxFilingRepository.save(filing); // write #1

    TaxFilingSubmittedEvent eventPayload = new TaxFilingSubmittedEvent(
        filing.getId(), filing.getEmployerId(), filing.getFilingPeriod(),
        filing.getTaxDue(), Instant.now());

    OutboxEvent outboxEvent = new OutboxEvent(
        "TaxFiling", filing.getId(), "TaxFilingSubmitted",
        toJson(eventPayload));
    outboxEventRepository.save(outboxEvent); // write #2

    return new TaxFilingResponse(filing.getId(), filing.getEmployerId(),
        filing.getFilingPeriod(), filing.getWagesReported(),
        filing.getTaxDue(), filing.getStatus().name());
}
```

Both writes go to the **same** database, inside the **same** @Transactional method. Postgres guarantees they both commit, or neither does — standard ACID behavior, nothing exotic. Notice there is **no Kafka call anywhere in this method**. Kafka is completely decoupled from the request.

A separate background job then does the actual delivery:

```
@Scheduled(fixedDelay = 5000)
public void publishUnpublishedEvents() {

    List<OutboxEvent> unpublished = outboxEventRepository.findByPublishedFalse();
    if (unpublished.isEmpty()) return;

    for (OutboxEvent event : unpublished) {
        try {
            kafkaTemplate.send(TOPIC, event.getAggregateId().toString(),
                event.getPayload().get());
        }
    }
}
```

```

        event.markPublished();
        outboxEventRepository.save(event);
    } catch (Exception e) {
        log.error("Failed to publish outbox event {}. Will retry next cycle.",
            event.getId(), e);
        // deliberately does NOT mark published – it must stay false so it is retried
    }
}
}

```

The single most important design decision here: **an outbox row is only ever marked published = true if the Kafka send genuinely succeeded**. If it fails, the row is simply left alone, and the next run of this scheduled method (five seconds later) will find it again and retry. Nothing is ever silently marked “done” unless it truly is.

A note on production-scale alternatives: the polling approach above is simple and correct, but at real production scale, most companies replace the poller with **Change Data Capture (CDC)**, typically via a tool called **Debezium**, which reads the database’s own internal write-ahead log directly for near-instant delivery with far less database query overhead. Polling was chosen deliberately here for its simplicity while learning the pattern; understanding this tradeoff is itself a useful thing to be able to discuss.

10.5 10.5 Apache Kafka Basics

In plain English: Kafka is like a shared logbook. One program writes entries into it (a **producer**). Any number of other programs can each read through that same logbook independently, at their own pace, keeping their own personal bookmark — one program reading an entry does not remove it or affect anyone else’s ability to read it too.

Technically, mapped to this project:

- **Topic:** tax-filing-events — the single topic both payment-service and notification-service subscribe to.
- **Producer:** tax-filing-service’s OutboxEventPublisher.
- **Consumers:** PaymentEventListener (in payment-service) and NotificationEventListener (in notification-service).
- **Consumer Groups:** payment-service-group and notification-service-group — two completely independent identities. Kafka tracks a separate offset for each group.

This was demonstrated directly during development: payment-service-group had been running for days and only saw new messages as they arrived. When notification-service-group connected for the very first time, with auto-offset-reset: earliest configured, Kafka correctly replayed the **entire history** of the topic — every filing ever submitted — to it in one burst, because that consumer group had no existing bookmark. Both groups then continued independently from that point forward.

A note on why event payloads are kept minimal and independently defined: payment-service and notification-service each define their **own copy** of TaxFilingSubmittedEvent (an identical Java record), rather than sharing one class between services. This is deliberate — sharing code across a service boundary quietly re-couples services that are supposed to be independent. Each consumer defines what it expects to receive, and the two sides only stay in sync through the agreed-upon JSON shape, not shared code.

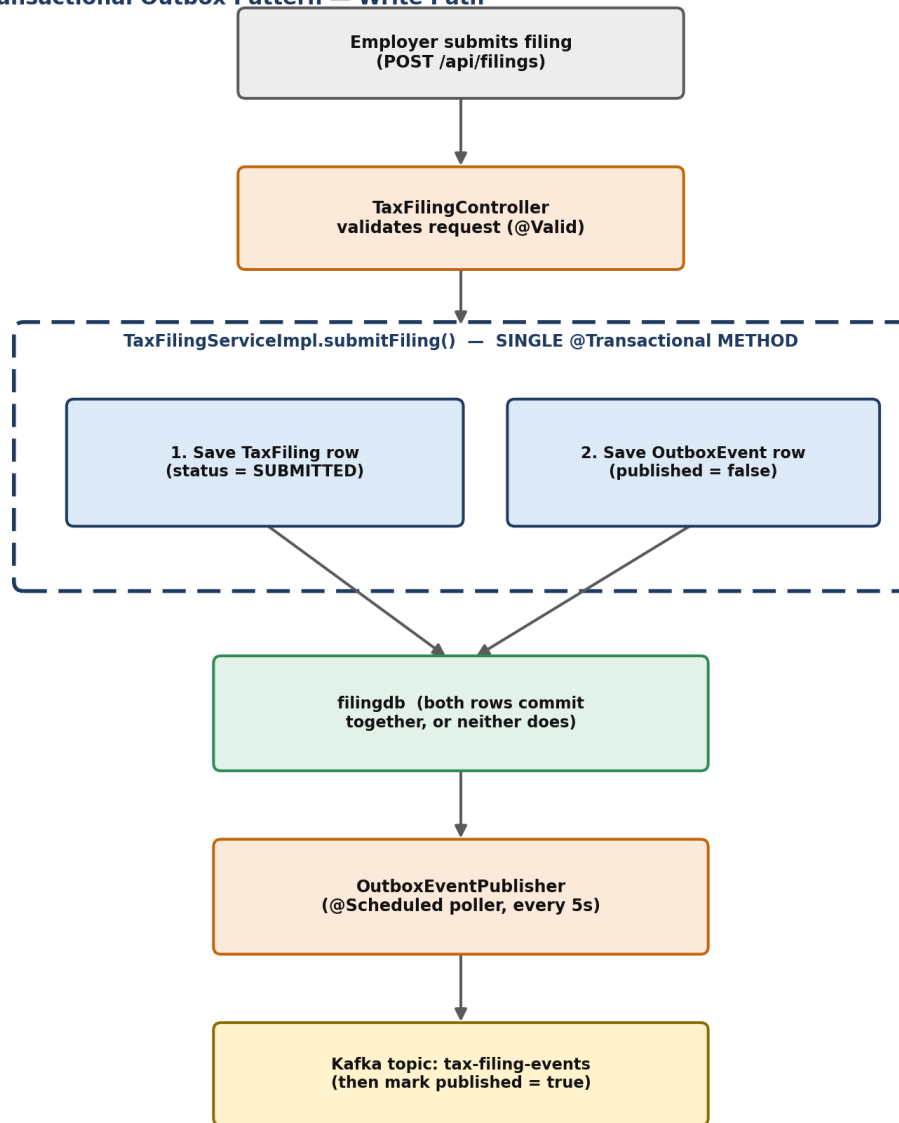
Figure 2. Transactional Outbox Pattern — Write Path

Figure 3: Figure 2. Transactional Outbox Pattern — Write Path

Figure 4. Two Independent Consumer Groups Reading the Same Topic

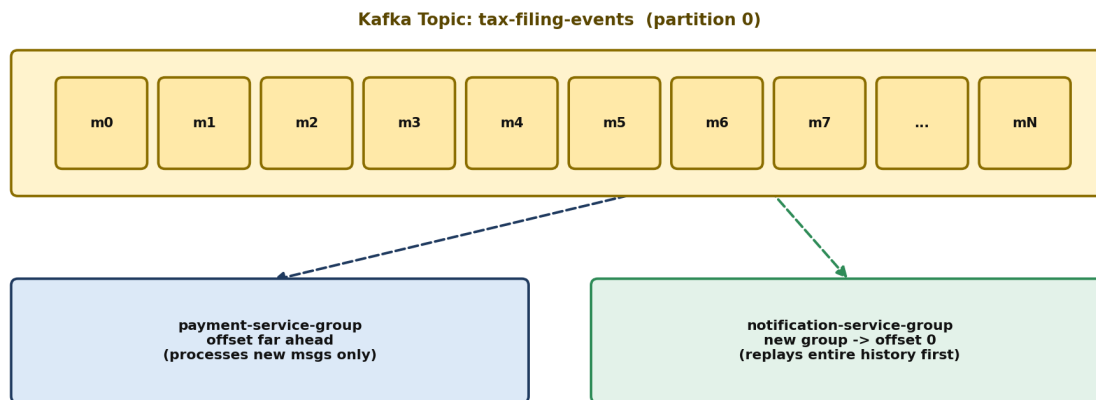


Figure 4: Figure 4. Two Independent Consumer Groups Reading the Same Topic

10.6 10.6 Idempotency

In plain English: Kafka’s delivery guarantee in this project is “at least once” — an event will always eventually arrive, but it might occasionally arrive twice (for example, if the outbox poller’s Kafka send actually succeeded, but the confirmation was lost on the way back, causing a safe retry). Idempotency means designing the consumer so that receiving the same event twice never results in doing the work twice.

Technically: The naive approach — if (exists) { skip } else { process } — has a real flaw: a race condition (formally, a “time-of-check to time-of-use” bug) where two near-simultaneous deliveries could both pass the check before either one finishes processing, resulting in a duplicate anyway. The fix implemented here lets the **database’s own primary key uniqueness** be the actual safety net, not application logic:

```
UUID eventId = event.filingId();

try {
    processedEventRepository.save(new ProcessedEvent(eventId));
} catch (DataIntegrityViolationException e) {
    log.info("Event {} already processed, skipping duplicate.", eventId);
    return;
}

// only reached if the insert above genuinely succeeded – a brand-new event
paymentGatewayInvoker.chargeGateway(event.filingId().toString());
Payment payment = new Payment(event.filingId(), event.employerId(), event.taxDue());
paymentRepository.save(payment);
```

`ProcessedEvent.eventId` is the primary key, with no auto-generation — it is set to exactly the incoming event’s own identifier. The very first insert attempt for a given ID either succeeds (genuinely new) or throws

`DataIntegrityViolationException` (already seen), and that outcome is atomic and race-condition-free at the database level, regardless of how many threads or service instances are involved. This was verified directly by manually resetting the Kafka consumer group’s offset back to the beginning of the topic (forcing every historical message to be redelivered) and confirming zero duplicate rows appeared in the payment table.

10.7 10.7 Circuit Breaker (Resilience4j)

In plain English: If you call a friend and they don’t pick up, calling them again immediately, ten times in a row, wastes your time and doesn’t help. A more sensible approach is: after a few unanswered attempts, stop calling for a while, then try again later to see if they’re available.

Technically: payment-service calls a deliberately unreliable simulated external payment gateway. Resilience4j’s `@CircuitBreaker` annotation wraps that call and tracks recent successes/failures in a sliding window. This project’s configuration:

```
resilience4j:
  circuitbreaker:
    instances:
      paymentGateway:
        sliding-window-size: 10
        minimum-number-of-calls: 5
        failure-rate-threshold: 50
        wait-duration-in-open-state: 10s
        permitted-number-of-calls-in-half-open-state: 3
        automatic-transition-from-open-to-half-open-enabled: true
```

This means: look at the last 10 calls; don’t judge anything until at least 5 have happened; if 50% or more failed, trip the circuit (“OPEN” — stop making real calls, fail immediately); after waiting 10 seconds, allow 3 test calls through (“HALF_OPEN”); if those look healthy enough, resume normal operation (“CLOSED”), otherwise trip open again.

This feature was implemented correctly on paper on the first attempt, but did not actually work — see Issue #13 in Section 12 for the full story of the bug (a classic Spring AOP self-invocation problem) and its fix, which is one of the most valuable lessons from this entire project.

10.8 10.8 Dead Letter Topic (DLT)

In plain English: If a piece of mail cannot be delivered after several attempts, a responsible postal service does not simply shred it — it gets set aside in a labeled “undeliverable” bin so a person can investigate later. A Dead Letter Topic is that bin, for Kafka messages.

Technically: By default, Spring Kafka retries a failed message a configured number of times, then logs an error and moves on — discarding the message entirely, with no trace left anywhere. This project replaces that default behavior with a `DefaultErrorHandler` wired to a `DeadLetterPublishingRecoverer`:

```
@Bean
public DefaultErrorHandler errorHandler(KafkaOperations<Object, Object> kafkaTemplate) {

    DeadLetterPublishingRecoverer recoverer = new
    ↪ DeadLetterPublishingRecoverer(kafkaTemplate,
        (record, exception) -> new TopicPartition(record.topic() + ".DLT",
        ↪ record.partition()));
```

Figure 3. Resilience4j Circuit Breaker — State Transitions

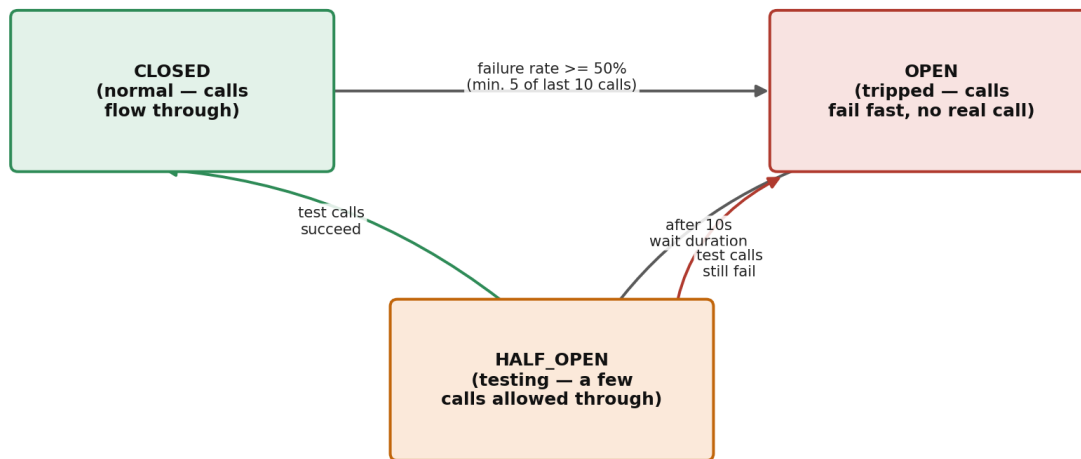


Figure 5: Figure 3. Resilience4j Circuit Breaker — State Transitions

```

FixedBackOff backOff = new FixedBackOff(1000L, 3L);

return new DefaultErrorHandler(recoverer, backOff);
}

```

After the configured retries (`FixedBackOff(1000L, 3L)` — 1 initial attempt plus 3 retries, 1 second apart) are exhausted, instead of being discarded, the message is automatically **republished** to a new topic, `tax-filing-events.DLT`, with the exact same key and payload. This was verified by temporarily forcing the simulated payment gateway to always fail, confirming the message appeared in the new `.DLT` topic in Kafka UI, and then reverting the forced failure.

10.9 JPA Auditing

In plain English: Instead of manually remembering to write “created on this date” or “last changed on this date” every single time a record is saved anywhere in the codebase, the framework can do this automatically, consistently, everywhere.

Technically: `tax-filing-service` defines a shared, reusable base class:

```

@MappedSuperclass
@EntityListeners(AuditingEntityListener.class)
public abstract class Auditable {

```

```

@CreatedDate
@Column(nullable = false, updatable = false)
private Instant createdAt;

@CreatedBy
@Column(nullable = false, updatable = false)
private String createdBy;

@LastModifiedDate
@Column(nullable = false)
private Instant updatedAt;

@LastModifiedBy
@Column(nullable = false)
private String updatedBy;
// getters omitted for brevity
}

```

TaxFiling extends Auditable, gaining all four columns automatically. @EnableJpaAuditing on the main application class switches this mechanism on. Since the project has no real login system yet, createdBy/updatedAt are filled by a placeholder AuditorAware<String> bean that always returns "system" — a normal, temporary stand-in that would be replaced with a real authenticated username once security is added.

payment-service's entities (Payment, ProcessedEvent) deliberately do **not** extend Auditable — they represent internal, system-generated bookkeeping rather than user-driven actions, so tracking “who” created them adds no real value.

10.10 10.10 Bean Validation

In plain English: Before any business logic runs, check that the incoming data actually makes sense (e.g., an amount cannot be negative, a required field cannot be blank) — and reject it immediately if not, rather than letting bad data quietly flow deeper into the system.

Technically:

```

public record TaxFilingRequest(
    @NotNull(message = "employerId is required")
    UUID employerId,

    @NotBlank(message = "filingPeriod is required")
    String filingPeriod,

    @NotNull @PositiveOrZero(message = "wagesReported cannot be negative")
    BigDecimal wagesReported,

    @NotNull @PositiveOrZero(message = "taxDue cannot be negative")
    BigDecimal taxDue
) {}

```

The @Valid annotation on the controller method is what actually activates these checks:

```
@PostMapping
public ResponseEntity<TaxFilingResponse> submitFiling(@Valid @RequestBody TaxFilingRequest
↪ request) {
    TaxFilingResponse response = taxFilingService.submitFiling(request);
    return ResponseEntity.status(HttpStatus.CREATED).body(response);
}
```

Without `@Valid`, none of the constraint annotations on `TaxFilingRequest` would have any effect at all — they would just be inert metadata.

11. Important Code Snippets

This section gathers the remaining key pieces of code not already shown in Section 10, with a short explanation under each.

11.1 The TaxFiling Entity

```
@Entity
@Table(name = "tax_filing")
public class TaxFiling extends Auditable {

    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;

    @Column(nullable = false)
    private UUID employerId;

    @Column(nullable = false)
    private String filingPeriod;

    @Column(nullable = false)
    private BigDecimal wagesReported;

    @Column(nullable = false)
    private BigDecimal taxDue;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private FilingStatus status;

    protected TaxFiling() {
        // required by Hibernate for reflection-based object creation
    }

    public TaxFiling(UUID employerId, String filingPeriod, BigDecimal wagesReported,
        ↪ BigDecimal taxDue) {
        this.employerId = employerId;
        this.filingPeriod = filingPeriod;
        this.wagesReported = wagesReported;
        this.taxDue = taxDue;
        this.status = FilingStatus.SUBMITTED;
    }

    // getters only – no setStatus()
    public void markProcessed() { this.status = FilingStatus.PROCESSED; }
    public void markFailed()    { this.status = FilingStatus.FAILED; }
}
```

Two design choices worth calling out: (1) the primary key uses `GenerationType.UUID`, generated in Java memory before the row is ever saved — this is exactly what lets the service layer build the matching `Out-boxEvent` in the same method, without needing an extra database round-trip to learn the new ID; (2) a protected no-argument constructor exists purely to satisfy Hibernate, while the real, public constructor

guarantees every TaxFiling object is created already in a fully valid state.

11.2 The OutboxEvent Entity

```
@Entity
@Table(name = "outbox_event")
public class OutboxEvent {

    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;

    @Column(nullable = false)
    private String aggregateType;

    @Column(nullable = false)
    private UUID aggregateId;

    @Column(nullable = false)
    private String eventType;

    @Column(nullable = false, columnDefinition = "TEXT")
    private String payload;

    @Column(nullable = false)
    private boolean published;

    @Column(nullable = false, updatable = false)
    private Instant createdAt;

    private Instant publishedAt;

    public OutboxEvent(String aggregateType, UUID aggregateId, String eventType, String
        ↪ payload) {
        this.aggregateType = aggregateType;
        this.aggregateId = aggregateId;
        this.eventType = eventType;
        this.payload = payload;
        this.published = false;
        this.createdAt = Instant.now();
    }

    public void markPublished() {
        this.published = true;
        this.publishedAt = Instant.now();
    }
}
```

payload uses columnDefinition = "TEXT" rather than the default VARCHAR, since JSON event bodies vary in size and should never risk silent truncation.

11.3 11.3 The Circuit Breaker Self-Invocation Fix

This pair of classes is the direct result of Issue #13 (Section 12) and is one of the most important patterns in the whole project — **the fix for a bug that would otherwise silently disable Spring’s circuit breaker.**

```
// PaymentGatewayInvoker.java – a separate Spring bean, deliberately
@Component
public class PaymentGatewayInvoker {

    private final PaymentGatewayClient paymentGatewayClient;

    public PaymentGatewayInvoker(PaymentGatewayClient paymentGatewayClient) {
        this.paymentGatewayClient = paymentGatewayClient;
    }

    @CircuitBreaker(name = "paymentGateway", fallbackMethod = "gatewayFallback")
    public void chargeGateway(String filingId) {
        paymentGatewayClient.charge(filingId);
    }

    public void gatewayFallback(String filingId, Throwable t) {
        throw new RuntimeException("Payment gateway unavailable, cannot process filing " +
            ↪ filingId, t);
    }
}
```

```
// PaymentEventListener.java – calls the invoker through dependency injection,
// never directly, so the call genuinely passes through Spring's proxy
@Component
public class PaymentEventListener {

    private final PaymentGatewayInvoker paymentGatewayInvoker;
    // ...other dependencies omitted...

    @KafkaListener(topics = "tax-filing-events", groupId = "payment-service-group")
    @Transactional
    public void handleTaxFilingSubmitted(String message) {
        // ...parse event, idempotency check...
        paymentGatewayInvoker.chargeGateway(event.filingId().toString());
        // ...save Payment...
    }
}
```

11.4 11.4 Custom Circuit Breaker State-Change Logging

By default, Resilience4j tracks circuit breaker state internally but does not print anything to the application log when the state changes. A small config class was added purely to make this visible for learning and demonstration purposes:

```
@Configuration
public class CircuitBreakerConfig {

    private final CircuitBreakerRegistry circuitBreakerRegistry;
```

```

public CircuitBreakerConfig(CircuitBreakerRegistry circuitBreakerRegistry) {
    this.circuitBreakerRegistry = circuitBreakerRegistry;
}

@PostConstruct
public void registerStateChangeLogging() {
    CircuitBreaker circuitBreaker =
↪ circuitBreakerRegistry.circuitBreaker("paymentGateway");
    circuitBreaker.getEventPublisher()
        .onStateTransition(event ->
            log.warn(">>> Circuit breaker 'paymentGateway' changed state: {} -> {}",
                event.getStateTransition().getFromState(),
                event.getStateTransition().getToState()));
}
}

```

11.5 11.5 application.yml Reference — Key Sections Per Service

tax-filing-service (full Kafka producer + Eureka client block):

```

spring:
  kafka:
    bootstrap-servers: localhost:9092
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.apache.kafka.common.serialization.StringSerializer
      acks: all
      properties:
        enable.idempotence: true

  eureka:
    client:
      service-url:
        defaultZone: http://localhost:8761/eureka/
    instance:
      prefer-ip-address: true
      hostname: localhost
      instance-id: ${spring.application.name}:${server.port}

```

payment-service (Kafka consumer + circuit breaker + DLT-relevant block):

```

spring:
  kafka:
    bootstrap-servers: localhost:9092
    consumer:
      group-id: payment-service-group
      auto-offset-reset: earliest
      key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
      value-deserializer: org.apache.kafka.common.serialization.StringDeserializer

  resilience4j:
    circuitbreaker:

```

```
instances:
  paymentGateway:
    sliding-window-size: 10
    minimum-number-of-calls: 5
    failure-rate-threshold: 50
    wait-duration-in-open-state: 10s
    permitted-number-of-calls-in-half-open-state: 3
    automatic-transition-from-open-to-half-open-enabled: true
```

api-gateway (routing configuration):

```
spring:
  cloud:
    gateway:
      server:
        webmvc:
          routes:
            - id: tax-filing-service
              uri: lb://tax-filing-service
              predicates:
                - Path=/tax-filing-service/**
              filters:
                - StripPrefix=1
            - id: payment-service
              uri: lb://payment-service
              predicates:
                - Path=/payment-service/**
              filters:
                - StripPrefix=1
```

eureka-server (the registry itself, configured to not register with itself):

```
eureka:
  client:
    register-with-eureka: false
    fetch-registry: false
  server:
    enable-self-preservation: false
```

12 12. Issues Encountered During Development, and How Each Was Resolved

This section is a faithful log of real problems hit while building this project, in the order they occurred. Each one follows the same structure: **what happened**, **why it happened**, and **how it was fixed** — because understanding the *diagnosis process* is often more valuable than the fix itself.

12.1 Issue 1 — Docker Desktop failed to start: WSL not installed

What happened: On first launching Docker Desktop on a Windows machine that had never used it before, it showed: “*Docker Desktop - WSL not installed.*”

Why: Docker Desktop on Windows requires the Windows Subsystem for Linux (WSL2) to actually run Linux containers. It had never been installed on this machine.

Fix: Ran `wsl --install` from an **administrator** PowerShell window, then restarted the machine (required — WSL2 does not become available until reboot). On first launch after rebooting, Ubuntu prompted for a new Linux username/password, which was created and never needed again directly. Docker Desktop then started cleanly.

12.2 Issue 2 — `docker-compose.yml` silently saved as `docker-compose.yml.txt`

What happened: After creating the compose file through a text editor, `docker compose up -d` failed to find any configuration.

Why: Windows Explorer hides known file extensions by default. The file’s true name was `docker-compose.yml.txt`, even though Explorer displayed it as `docker-compose.yml`.

Fix: Ran `dir` in PowerShell (which always shows the true filename, unlike Explorer), confirmed the hidden `.txt`, and corrected it with `Rename-Item "docker-compose.yml.txt" "docker-compose.yml"`.

12.3 Issue 3 — `application.yml` never actually existed; services silently used defaults

What happened: `tax-filing-service` started on the wrong port (8080 instead of the configured 8081) with no database connection at all, despite `application.yml` apparently being “created” in an earlier step.

Why: Only the default `application.properties` stub (auto-generated by Spring Initializr) actually existed in `src/main/resources`; the intended `application.yml` file had never been created, so Spring Boot silently fell back to every default value.

Fix: Deleted `application.properties`, created `application.yml` directly via a PowerShell here-string (Out-File), and verified its contents with `type application.yml` before restarting the application. From this point forward, every new service’s configuration was verified with `type` immediately after creation, to catch this exact mistake early.

12.4 Issue 4 — Eclipse showed `pom.xml` errors that were not real errors

What happened: Every `pom.xml` in the project displayed two red errors in Eclipse’s Problems view: `cvc-elt.1.a: Cannot find the declaration of element 'project'` and `Downloading external resources is disabled`.

Why: These come from Eclipse’s XML editor trying to validate the file against Maven’s official XML schema, which it fetches over the internet. With that download blocked (by an Eclipse preference or network policy), Eclipse simply cannot *verify* the file — this says nothing about whether Maven itself can actually build it.

Fix: Confirmed harmless by running an actual Maven build directly (**Run As** → **Maven Build...** → **Goals: compile**) and getting BUILD SUCCESS. From then on, this specific pair of warnings was treated as known-and-ignorable on every project.

12.5 Issue 5 — Stale processes held onto ports after a service was “stopped”

What happened: Restarting a service repeatedly failed with Web server failed to start. Port 8081 was already in use.

Why: A previous run’s Java process had not fully terminated, and was still holding the port.

Fix:

```
netstat -ano | findstr :8081
taskkill /PID <the_pid_shown> /F
```

(run from an administrator PowerShell window when access was denied to a non-elevated session).

12.6 Issue 6 — TaxFiling entity was missing its constructor and getters

What happened: After building the entity’s fields across several incremental steps, a later class (TaxFilingServiceImpl) failed to compile with a wall of errors: “*The constructor TaxFiling(UUID, String, BigDecimal, BigDecimal) is undefined,*” “*The method getId() is undefined,*” and five more like it.

Why: The constructor and getter methods had simply never been added — the entity had fields but no way to construct it or read its values. Java reports these errors at the *call site*, which can be misleading: the errors appeared on TaxFilingServiceImpl, even though the actual missing code lived in an entirely different file, TaxFiling.

Fix: Reviewed the actual entity source directly (rather than only the error messages) and added the missing protected no-arg constructor, the public business constructor, and all six getters.

12.7 Issue 7 — jakarta.validation imports unresolved

What happened: TaxFilingRequest.java showed unresolved import errors for @NotNull, @NotBlank, and @PositiveOrZero.

Why: The spring-boot-starter-validation dependency had never been added to pom.xml — Bean Validation is a separate Spring Boot starter from Web or JPA, not bundled automatically.

Fix: Added the dependency, then **Maven** → **Update Project** in Eclipse to force dependency re-resolution.

12.8 Issue 8 — YAML indentation mistake silently broke a new config section

What happened: After pasting a new Resilience4j configuration block into application.yml, payment-service compiled and ran, but the settings appeared to have no effect at all.

Why: The pasted block had landed nested three levels too deep — as if it were a child of `management.endpoints.web.include` — rather than as a new top-level section, a sibling of `spring`, `server`, and `management`. YAML uses indentation to define structure, so this silently changed what the file *meant*, without producing any error.

Fix: Rewrote the entire file from scratch via a PowerShell here-string with deliberate, explicit indentation, and verified the result with `type` before restarting.

12.9 Issue 9 — Spring Cloud version incompatible with Spring Boot version

What happened: Adding `spring-cloud-starter-netflix-eureka-client` to `tax-filing-service` (Spring Boot 3.5.16) produced a real dependency resolution error in Eclipse — distinct from the harmless XML warning in Issue #4.

Why: `pom.xml` had been given `spring-cloud.version: 2025.1.2`, which is the release train intended for Spring Boot 4.x, and also had **no** `<dependencyManagement>` BOM import block at all.

Fix: Added the missing `<dependencyManagement>` block importing `spring-cloud-dependencies`, and corrected the version to `2025.0.0` — the release train actually compatible with Spring Boot 3.5.x. `payment-service` (Spring Boot 4.1.0) correctly remained on `2025.1.2`.

12.10 Issue 10 — PowerShell silently stripped Spring placeholders from a config value

What happened: After adding `instance-id: ${spring.application.name}:${server.port}` to an `application.yml` created via a PowerShell script, the application failed to start with a YAML parser error: *“mapping values are not allowed here... instance-id: :”*

Why: The file had been generated using a **double-quoted** PowerShell here-string (`@" . . . "@`). PowerShell performs variable interpolation inside double-quoted strings, including here-strings, and tried to interpret `${spring.application.name}` as its own variable reference. Finding nothing matching, it silently substituted an empty string, corrupting the line into `instance-id: :` — invalid YAML.

Fix: Switched to a **single-quoted** here-string (`@' . . . '@`), which PowerShell treats as fully literal text with zero interpolation, preserving the `${ . . . }` placeholders exactly as written for Spring Boot itself to resolve at runtime.

12.11 Issue 11 — Eureka registered services under an illegal hostname

What happened: Requests routed through `api-gateway` failed with `java.net.URISyntaxException: Illegal character in hostname at index 12: http://Vinay_Laptop.mshome.net:8081/...`

Why: The Windows machine’s hostname, `Vinay_Laptop`, contains an underscore — a character that is technically illegal in a URI hostname per RFC 3986. Java’s strict URI parser rejected it outright when the gateway tried to build a request to the address Eureka had returned. Setting `eureka.instance.prefer-ip-address: true` alone did not fix this, because the registered **instance-id** (used to build the clickable/dashboard address) is derived from `spring.cloud.client.hostname`, a separate mechanism from `eureka.instance.hostname`.

Fix: Added both `eureka.instance.hostname: localhost` **and** an explicit `eureka.instance.instance-id: ${spring.application.name}:${server.port}` override to every client service’s configuration, bypassing the problematic OS hostname entirely. Confirmed fixed by checking the Eureka dashboard showed clean addresses like `tax-filing-service:8081` with no hostname at all.

12.12 Issue 12 — notification-service failed to register with Eureka

What happened: On startup, notification-service threw `NoSuchBeanDefinitionException: No qualifying bean of type 'com.netflix.discovery.shared.transport.jersey.TransportClientFactories'`.

Why: notification-service was the only service in the project without spring-boot-starter-web on its classpath (it exposes no REST endpoints of its own, so it seemed unnecessary). However, the Eureka client internally relies on Spring Web's HTTP client infrastructure (`RestTemplate`) to actually register and send heartbeats — without it, Spring Cloud cannot fully wire up the Eureka client's transport layer.

Fix: Added spring-boot-starter-web even though the service has no controllers of its own — it is required purely to support the Eureka client dependency.

12.13 Issue 13 — The circuit breaker was configured correctly but never actually engaged

What happened: Despite the `@CircuitBreaker` annotation, correct Resilience4j configuration, and a working fallback method, `/actuator/circuitbreakerevents` always returned an empty list, no matter how many times the simulated payment gateway failed.

Why: This is a classic Spring AOP **self-invocation** problem. `PaymentEventListener.handleTaxFilingSubmitted()` called `chargeGateway()` — a method defined in the *same class*, annotated `@CircuitBreaker` — directly on this. Spring implements annotations like `@CircuitBreaker` (and `@Transactional`, `@Cacheable`) by wrapping the real object in a proxy; that proxy only intercepts calls made *from outside the class*. A call from within the same class bypasses the proxy entirely, silently disabling the annotation with no error or warning of any kind.

Fix: Extracted `chargeGateway()` and its fallback method into a brand-new Spring bean, `PaymentGatewayInvoker` (Section 11.3), and updated `PaymentEventListener` to call it through dependency injection instead. This turns the call into a genuine cross-bean call that correctly passes through Spring's proxy. Verified fixed by watching real `CircuitBreaker` 'paymentGateway' is `OPEN` exceptions and full `CLOSED` → `OPEN` → `HALF_OPEN` → `CLOSED` state-transition logs appear for the first time.

12.14 Issue 14 — A two-part ObjectMapper / date deserialization bug

What happened (part one): payment-service failed to deserialize incoming Kafka messages: `InvalidDefinitionException: Java 8 date/time type java.time.Instant not supported by default`.

Why (part one): `pom.xml` contained a typo — `spring-boot-starter-webmvc` instead of the real artifact, `spring-boot-starter-web`. This meant Spring Boot's usual auto-configured `ObjectMapper` (which correctly supports `Instant` out of the box) was never created. A defensive manual `ObjectMapper` bean was added as a stop-gap.

What happened (part two): After fixing the `pom.xml` typo, the exact same `Instant` deserialization error reappeared.

Why (part two): The manual `ObjectMapper` bean added as a stop-gap (`new ObjectMapper()`, with no modules registered) now **overrode** Spring Boot's correctly auto-configured one, reintroducing the same problem from a different cause.

Fix: Corrected the `pom.xml` typo **and** updated the manual bean to explicitly register the required module:

```
@Bean
public ObjectMapper objectMapper() {
```

```
    ObjectMapper mapper = new ObjectMapper();  
    mapper.registerModule(new JavaTimeModule());  
    return mapper;  
}
```

12.15 Issue 15 — Permanently-failing messages were silently discarded

What happened: While debugging the circuit breaker, console logs periodically showed `Backoff Fixed-BackOffExecution[...] exhausted for tax-filing-events-0@N` — and the corresponding filing was never processed, with absolutely no other trace of it anywhere.

Why: This was default, expected Spring Kafka behavior at the time: after a message’s configured retry attempts are exhausted, the framework logs an error and simply moves on to the next message. The failed message is gone.

Fix: Built a Dead Letter Topic (Section 10.8) — a `DefaultErrorHandler` bean wired with a `DeadLetterPublishingRecoverer`, which republishes any message that exhausts retries to a new topic, `tax-filing-events.DLT`, instead of discarding it. Verified by temporarily forcing the simulated gateway to fail 100% of the time, confirming the message correctly appeared in Kafka UI under the new `.DLT` topic, then reverting the forced failure back to its normal ~90% random rate.

13. Testing Strategy — How Each Feature Was Verified

No automated test suite (JUnit, Testcontainers, etc.) was written in this phase of the project — this is explicitly listed as a Future Improvement in Section 14. All verification was manual and exploratory, but was still deliberate and repeatable. Four tools were used together for every test:

1. **Postman** — to trigger the system (send POST requests).
2. **The Eclipse Console** — to watch the actual application log output line-by-line as a request flowed through the system, including Hibernate’s raw SQL (enabled via `show-sql: true`) and every custom log statement.
3. **DBeaver** — to visually confirm the *actual database state* after a test, not just trust the logs.
4. **Kafka UI** (<http://localhost:8090>) — to visually confirm topics, message counts, message contents, and consumer group state.

13.1 Basic end-to-end flow

A filing was submitted via Postman directly to `tax-filing-service` (POST `http://localhost:8081/api/filings`), and separately, through the gateway (POST `http://localhost:8080/tax-filing-service/api/filings`), confirming both paths produced an identical 201 Created response and identical downstream behavior.

13.2 Outbox pattern

After a successful submission, `outbox_event` was checked in DBeaver — confirming a new row existed with `published = false`. Within five seconds (the poller’s schedule), the row was re-checked and confirmed to have flipped to `published = true`, with `published_at` populated. Kafka UI was then checked to confirm a matching message existed in the `tax-filing-events` topic.

13.3 Multiple independent consumers

`payment-service` and `notification-service` were both run simultaneously, and a single filing submission was confirmed to be independently logged/processed by **both** services — proving Kafka’s consumer group model correctly fans out one event to multiple, unrelated subscribers.

13.4 Idempotency

`payment-service` was stopped, and its consumer group’s offset was manually reset to the beginning of the topic using the Kafka command-line tool (run inside the Kafka Docker container):

```
docker exec -it kafka kafka-consumer-groups --bootstrap-server localhost:9092 --group
↪ payment-service-group --topic tax-filing-events --reset-offsets --to-earliest --execute
```

`payment-service` was then restarted, forcing it to re-read every historical message on the topic — including many it had already successfully processed. The console showed the expected `Event {}` already processed, skipping duplicate. message for each previously-seen event, and DBeaver confirmed **no duplicate rows** appeared in the `payment` table.

13.5 Circuit breaker

The simulated gateway’s random failure rate was temporarily increased (from the normal ~90% failure to as close to guaranteed failure as needed for a reliable test), and 5–8 filings were submitted in quick succession. The following were all confirmed together:

- The console showed real CircuitBreaker 'paymentGateway' is OPEN and does not permit further calls exceptions.
- The custom state-transition logger (Section 11.4) printed the full CLOSED → OPEN → HALF_OPEN → CLOSED cycle in real time.
- GET `http://localhost:8082/actuator/circuitbreakerevents` returned a populated array of real events (previously always empty before the fix in Issue #13).
- GET `http://localhost:8082/actuator/circuitbreakers` showed the live current state and failure-rate metrics.

13.6 13.6 Dead Letter Topic

`PaymentGatewayClient.charge()` was temporarily hardcoded to always throw an exception (100% failure, removing randomness from the test). A single filing was submitted, the console showed exactly four failed attempts (1 initial + 3 retries, per the configured `FixedBackOff(1000L, 3L)`), and Kafka UI was checked immediately afterward — confirming a **new topic**, `tax-filing-events.DLT`, had been automatically created, containing exactly one message: the failed filing's event. The forced 100% failure was then reverted.

13.7 13.7 Service discovery and gateway routing

After every configuration fix in Issues #9–#12, the Eureka dashboard (`http://localhost:8761`) was used as the primary verification tool — confirming each of the five services showed status UP with a clean address (no illegal hostname characters), before attempting any gateway routing test.

14 14. Project Status and Future Improvements

14.1 14.1 What is fully working and verified

- All five services build, start, and register correctly with Eureka.
- The API Gateway correctly routes requests to both `tax-filing-service` and `payment-service` by service name, without any hardcoded ports.
- The full transactional outbox write path is implemented and verified (Section 13.2).
- Kafka correctly fans out one event to two independent, unrelated consumer groups (Section 13.3).
- Idempotent processing is implemented and verified under a forced-duplicate scenario (Section 13.4).
- The circuit breaker is correctly wired, engages under real failure conditions, and completes the full state-transition cycle (Section 13.5).
- A Dead Letter Topic correctly captures messages that exhaust all retries, instead of silently discarding them (Section 13.6).

14.2 14.2 Known limitations and suggested future work

- **No automated test suite.** All verification described in Section 13 was manual. A natural next step would be adding JUnit + Mockito unit tests for the service layer, and integration tests using **Testcontainers** to spin up real (ephemeral) Postgres and Kafka instances for each test run.
- **No centralized/global exception handling.** Validation failures and other errors currently return Spring's default error responses rather than a deliberately designed, consistent JSON error shape (a `@RestControllerAdvice`-based global exception handler was planned early in the project's roadmap but not yet implemented).
- **No authentication or authorization.** `AuditorAwareImpl` always returns the placeholder "system" for `createdBy/updatedBy`; there is no real login system, and no endpoint requires any credentials.
- **The outbox poller uses fixed-interval polling, not Change Data Capture.** As discussed in Section 10.4, a production system at real scale would typically replace this with Debezium reading the database's write-ahead log directly, for lower latency and less database load.
- **No containerization of the application services themselves.** Currently, only the shared infrastructure (Postgres, Kafka, Zookeeper, Kafka UI) runs in Docker; all five Spring Boot services are run manually from Eclipse. Writing a `Dockerfile` for each service and adding them to `docker-compose.yml` would allow the entire system to start with a single command.
- **No API documentation (OpenAPI/Swagger).** Endpoints are currently only documented in this file and tested manually via Postman.

15 15. How to Run This Project Locally — Quick Reference Runbook

1. Ensure Docker Desktop is running.

2. Start the shared infrastructure:

```
cd C:\uitax\infra
docker compose up -d
docker compose ps
```

3. Confirm Kafka UI is reachable at <http://localhost:8090> and both Postgres containers are healthy.

4. In Eclipse, start each service **in this order**, waiting for each to fully finish starting before moving to the next:

1. eureka-server — confirm <http://localhost:8761> loads.
2. tax-filing-service
3. payment-service
4. notification-service
5. api-gateway

5. Confirm all five services show status UP on the Eureka dashboard.

6. In Postman, send a test request:

```
POST http://localhost:8080/tax-filing-service/api/filings
Content-Type: application/json
```

```
{
  "employerId": "3f7b1a2e-4c5d-4e6f-8a9b-1c2d3e4f5a6b",
  "filingPeriod": "2026-Q3",
  "wagesReported": 60000.00,
  "taxDue": 5200.00
}
```

7. Confirm a 201 Created response, then watch the payment-service and notification-service consoles for the event being independently processed within a few seconds.

16 16. Appendix

16.1 16.1 Full docker-compose.yml (Shared Infrastructure)

```
services:
  zookeeper:
    image: confluentinc/cp-zookeeper:7.6.1
    container_name: zookeeper
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181
      ZOOKEEPER_TICK_TIME: 2000
    ports:
      - "2181:2181"

  kafka:
    image: confluentinc/cp-kafka:7.6.1
    container_name: kafka
    depends_on:
      - zookeeper
    ports:
      - "9092:9092"
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: PLAINTEXT:PLAINTEXT,PLAINTEXT_INTERNAL:PLAINTEXT
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092,PLAINTEXT_INTERNAL://kafka:29092
      KAFKA_LISTENERS: PLAINTEXT://0.0.0.0:9092,PLAINTEXT_INTERNAL://0.0.0.0:29092
      KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT_INTERNAL
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1

  kafka-ui:
    image: provectuslabs/kafka-ui:latest
    container_name: kafka-ui
    depends_on:
      - kafka
    ports:
      - "8090:8080"
    environment:
      KAFKA_CLUSTERS_0_NAME: local
      KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS: kafka:29092
      KAFKA_CLUSTERS_0_ZOOKEEPER: zookeeper:2181

  postgres-filing:
    image: postgres:16
    container_name: postgres-filing
    environment:
      POSTGRES_DB: filingdb
      POSTGRES_USER: filinguser
      POSTGRES_PASSWORD: filingpass
    ports:
      - "5433:5432"
    volumes:
      - filing-data:/var/lib/postgresql/data
```

```
postgres-payment:
  image: postgres:16
  container_name: postgres-payment
  environment:
    POSTGRES_DB: paymentdb
    POSTGRES_USER: paymentuser
    POSTGRES_PASSWORD: paymentpass
  ports:
    - "5434:5432"
  volumes:
    - payment-data:/var/lib/postgresql/data

volumes:
  filing-data:
  payment-data:
```

Explaining the two Kafka listeners, briefly: Kafka advertises itself two different ways at once — `localhost:9092` for clients outside Docker (the Eclipse-run Spring Boot services on the Windows host) and `kafka:29092` for clients inside the Docker network (Kafka UI). Containers resolve each other by container name, but the Windows host cannot resolve `kafka` as a hostname — only `localhost` — which is why both listeners are necessary.

16.2 16.2 Full `application.yml` — `tax-filing-service`

```
server:
  port: 8081

spring:
  application:
    name: tax-filing-service

  datasource:
    url: jdbc:postgresql://localhost:5433/filingdb
    username: filinguser
    password: filingpass
    driver-class-name: org.postgresql.Driver

  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
    properties:
      hibernate:
        format_sql: true
    open-in-view: false

  kafka:
    bootstrap-servers: localhost:9092
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.apache.kafka.common.serialization.StringSerializer
```

```
    acks: all
    properties:
      enable.idempotence: true

management:
  endpoints:
    web:
      exposure:
        include: health,info

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/
  instance:
    prefer-ip-address: true
    hostname: localhost
    instance-id: ${spring.application.name}:${server.port}
```

16.3 16.3 Full application.yml — payment-service

```
server:
  port: 8082

spring:
  application:
    name: payment-service

  datasource:
    url: jdbc:postgresql://localhost:5434/paymentdb
    username: paymentuser
    password: paymentpass
    driver-class-name: org.postgresql.Driver

  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
    properties:
      hibernate:
        format_sql: true
    open-in-view: false

  kafka:
    bootstrap-servers: localhost:9092
    consumer:
      group-id: payment-service-group
      auto-offset-reset: earliest
      key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
      value-deserializer: org.apache.kafka.common.serialization.StringDeserializer

management:
```

```
endpoints:
  web:
    exposure:
      include: health,info,circuitbreakers,circuitbreakerevents

resilience4j:
  circuitbreaker:
    instances:
      paymentGateway:
        sliding-window-size: 10
        minimum-number-of-calls: 5
        failure-rate-threshold: 50
        wait-duration-in-open-state: 10s
        permitted-number-of-calls-in-half-open-state: 3
        automatic-transition-from-open-to-half-open-enabled: true

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/
  instance:
    prefer-ip-address: true
    hostname: localhost
    instance-id: ${spring.application.name}:${server.port}
```

16.4 16.4 Full application.yml — notification-service

```
server:
  port: 8083

spring:
  application:
    name: notification-service
  kafka:
    bootstrap-servers: localhost:9092
    consumer:
      group-id: notification-service-group
      auto-offset-reset: earliest
      key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
      value-deserializer: org.apache.kafka.common.serialization.StringDeserializer

management:
  endpoints:
    web:
      exposure:
        include: health,info

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/
  instance:
```



```
prefer-ip-address: true
hostname: localhost
instance-id: ${spring.application.name}:${server.port}
```

16.5 16.5 Full application.yml — api-gateway

```
server:
  port: 8080

spring:
  application:
    name: api-gateway
  cloud:
    gateway:
      server:
        webmvc:
          routes:
            - id: tax-filing-service
              uri: lb://tax-filing-service
              predicates:
                - Path=/tax-filing-service/**
              filters:
                - StripPrefix=1
            - id: payment-service
              uri: lb://payment-service
              predicates:
                - Path=/payment-service/**
              filters:
                - StripPrefix=1

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/
  instance:
    prefer-ip-address: true
    hostname: localhost
    instance-id: ${spring.application.name}:${server.port}
```

16.6 16.6 Full application.yml — eureka-server

```
server:
  port: 8761

spring:
  application:
    name: eureka-server

eureka:
  client:
    register-with-eureka: false
```

```
fetch-registry: false
server:
  enable-self-preservation: false
```

17 Closing Note

Every diagram, code snippet, configuration value, and issue described in this document reflects the actual system as built and verified, not an idealized version of it. The fifteen issues in Section 12 are, in a real sense, the most valuable part of this document — they are the specific, concrete evidence of hands-on debugging experience with WSL/Docker setup, Spring configuration, YAML, Maven dependency management, Spring Cloud version compatibility, Spring AOP proxy behavior, and Kafka’s delivery and consumer-group semantics. This project is intended to keep growing — the Future Improvements listed in Section 14 are the natural next steps.